Practical: 9

Practical 9: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing
Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?

Yes. In the registration script, password_hash($password, PASSWORD_DEFAULT) is used to securely hash passwords using the bcrypt algorithm before performing the database insert. The raw plain-text password is never stored in the database.

2. Is duplicate email/username checked?

Yes. Prior to database insertion, a prepared SELECT statement queries 'SELECT id FROM users WHERE email = ? OR username = ?'. If any matching record is found (num_rows > 0), the registration is rejected and an appropriate error message is displayed.

3. Are frontend and backend validations consistent?

Yes. Both client-side JavaScript and server-side PHP enforce the same rules: verifying required fields are not empty, validating email format, ensuring minimum password length (at least 6 characters), and checking password confirmation match. Client-side validation provides immediate UX feedback, while backend validation guarantees data security.

B. Implementation / Output

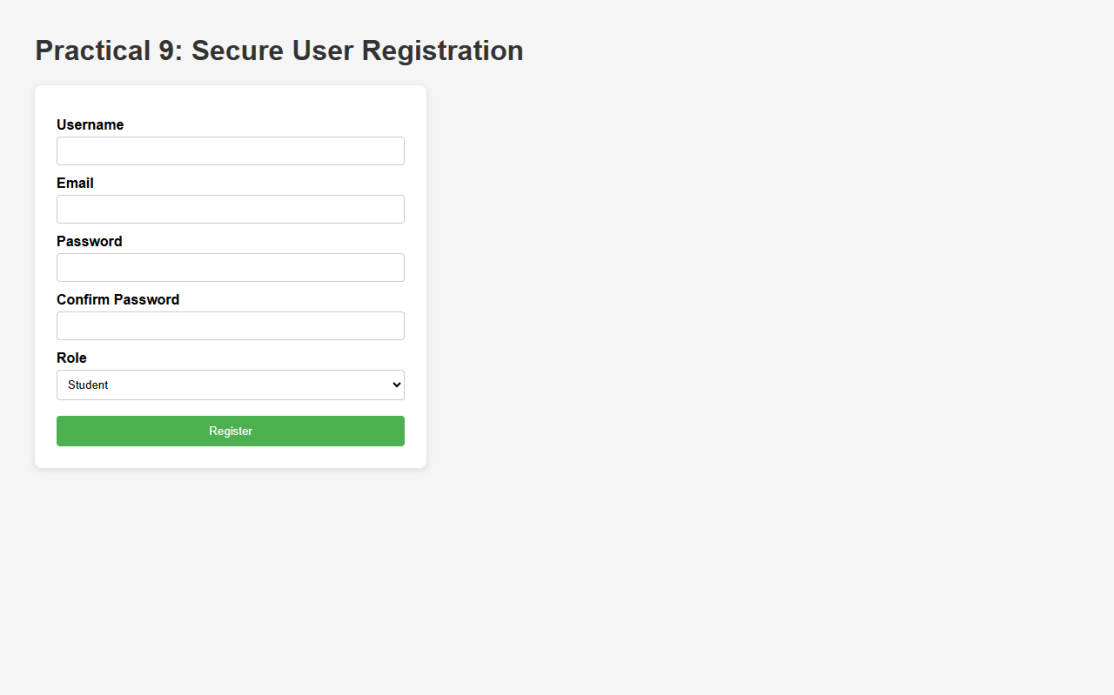


Figure 9.1: Secure User Registration Interface (Input Form with Validation Controls)

C. Screenshots

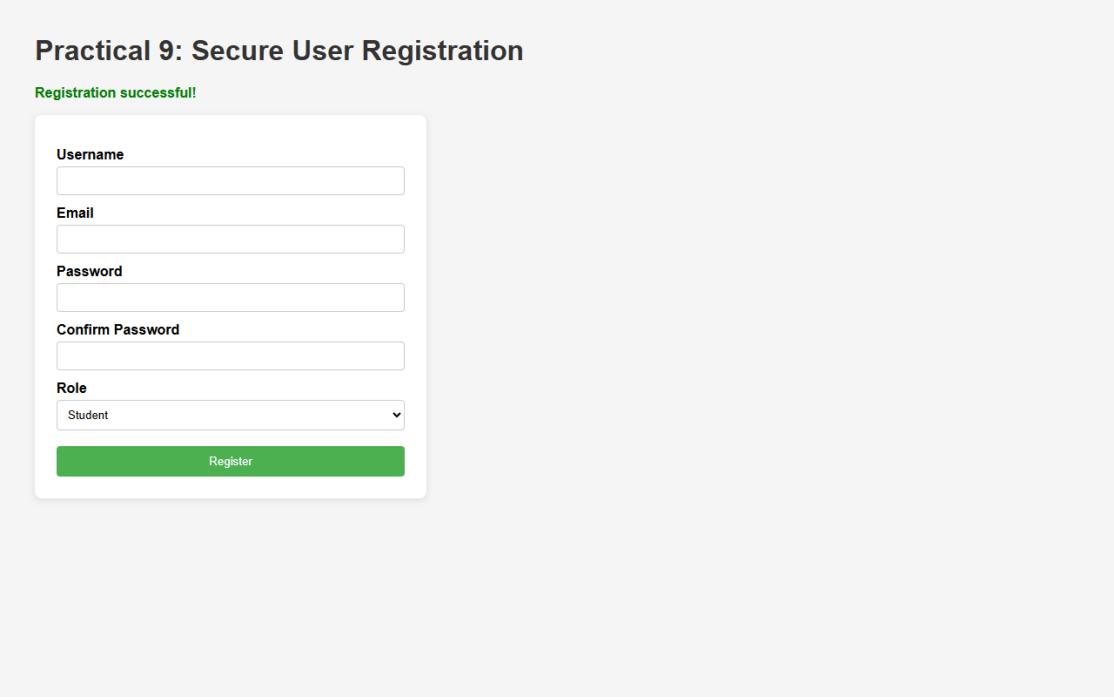


Figure 9.2: Successful User Registration with Server-side Insertion Message

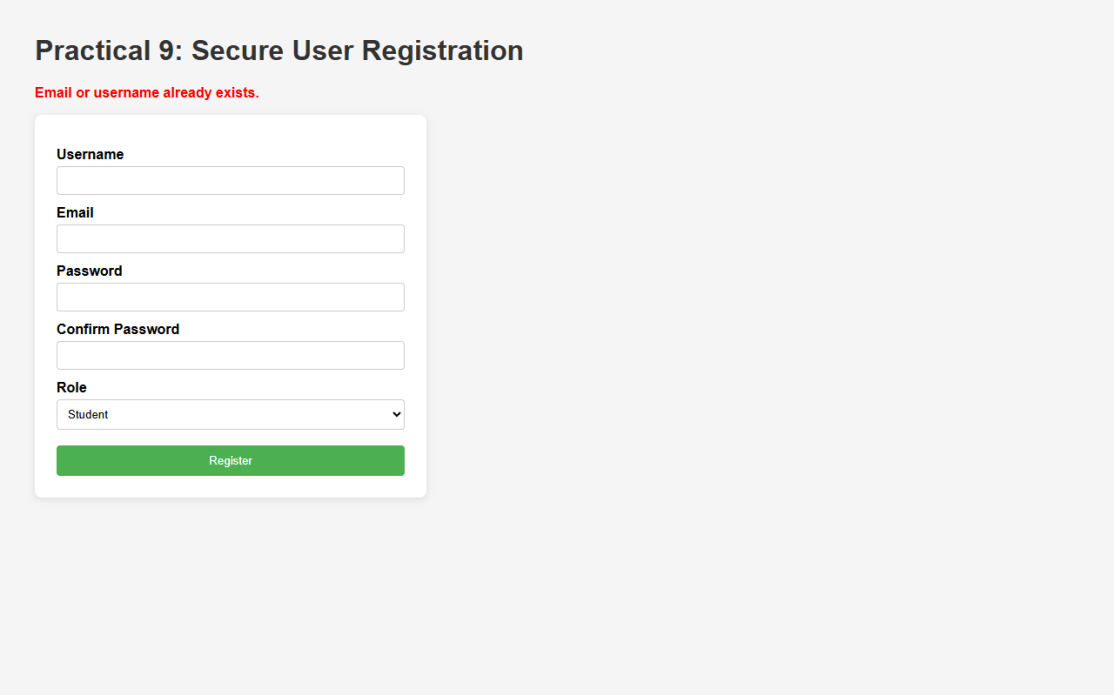


Figure 9.3: Duplicate Email and Username Check Preventing Redundant Account Creation

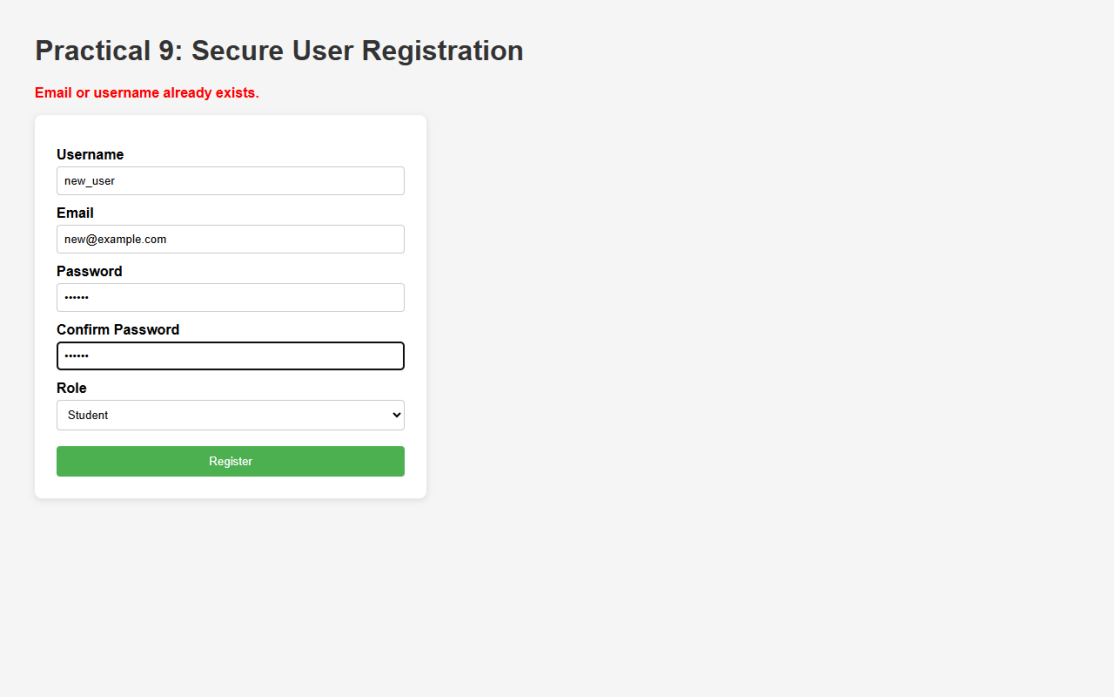


Figure 9.4: Frontend and Backend Password Validation Check

D. GitHub Link

https://github.com/VRAJ-710/studenthub-portal

E. Challenges Faced

- Ensuring synchronization between client-side JavaScript validation and server-side PHP validation without discrepancies.
- Handling duplicate constraint violations gracefully with custom user-friendly error messages rather than raw database error codes.
- Preventing SQL injection vulnerabilities by migrating database operations to MySQLi prepared statements.

F. Learning Outcomes

- Implemented secure user registration using MySQLi prepared statements.
- Learned secure password hashing techniques using password_hash() with the bcrypt algorithm.
- Understood how to prevent duplicate user accounts by performing pre-insertion database validation checks.

Faculty Remarks: